INDIAN INSTITUTE OF INFORMATION & COMMUNICATION TECHNOLOGY

(I I C T)

SUMMER INTERNSHIP PROGRAM

2026

Internship Assignment Portfolio — Data Science & Machine Learning with the Titanic Dataset

Name	Vishnu Thangaraj
Enrollment Number	845488
Email ID	vishnuthangaraj@proton.me
Contact	9385884548

Complete assignment code is available in the GitHub repository:

https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026

TABLE OF CONTENTS

ASSIGNMENT 1

Date: 15 June 2026

Task: Download Anaconda Jupyter Notebook and run a simple code.

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment 1/Setup and Installation.ipynb

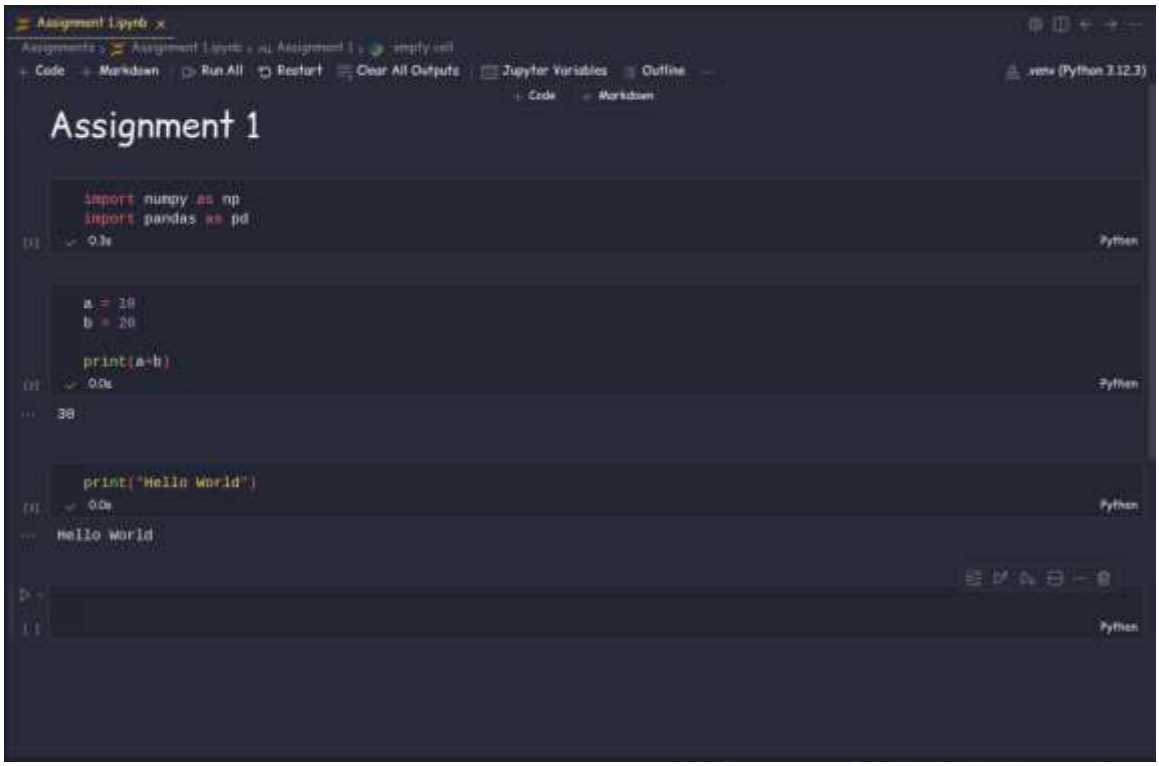


Figure 1.1 — Anaconda / Jupyter Notebook setup and test run

ASSIGNMENT 2

Date: 17 June 2026

1. Conceptual Questions

Question 1: Best chart to visualize survival by gender and class

To compare **survival by both gender and passenger class**, the best choice is a **grouped (clustered) bar chart**.

Why it works best:

- The x-axis shows passenger class (1st, 2nd, 3rd).
- Each class contains two bars: one for Male and one for Female.
- The y-axis shows either Survival rate (%) (recommended), or Number of survivors.

This makes it easy to compare:

- Male vs. female survival within each class.
- Survival differences across classes.
- The interaction between the two variables in a single view.

For example, the chart would typically reveal the well-known pattern:

- Female passengers had much higher survival rates than males.
- First-class passengers had higher survival rates than second- and third-class passengers.
- Third-class males had the lowest survival rates.

Question 2: How poor design choices distort a visualization's message

Poor visualization design can mislead viewers, causing them to draw incorrect conclusions even when the underlying data is accurate. Common design mistakes and their effects include:

1. **Misleading scales** — Starting the y-axis above zero in a bar chart can exaggerate small differences; uneven axis intervals can make trends appear stronger or weaker than they really are. Effect: viewers may overestimate or underestimate the magnitude of changes.
2. **Excessive or inappropriate color** — Too many bright colors make a chart cluttered and distracting; inconsistent colors for the same category confuse readers. Effect: important patterns become harder to identify.
3. **3D effects** — Three-dimensional bars or pie charts distort the perceived size of values, since objects closer to the viewer appear larger. Effect: comparisons between categories become inaccurate.
4. **Improper chart selection** — Using a pie chart for many categories, or a line chart for unrelated categories, reduces readability and makes relationships difficult to interpret.
5. **Overcrowding** — Too many labels, legends, or data points make the visualization difficult to read, and the key message gets lost in unnecessary detail.

6. **Missing labels or context** — Omitting axis labels, units, titles, or legends leaves the audience unsure of what the chart represents, so it can be misunderstood.

Example

Imagine a bar chart showing Titanic survival rates:

- If the y-axis starts at 80% instead of 0%, the difference between first-class and second-class survival may appear much larger than it actually is.
- If every bar is a different bright color without explanation, viewers may assume the colors have meaning when they do not.

Best Practices

- Use honest and consistent axis scales.
- Limit colors to highlight important information.
- Choose the chart type that best fits the data.
- Keep the design simple and uncluttered.
- Include clear titles, labels, legends, and units.

A well-designed visualization helps viewers understand the data accurately, while poor design can unintentionally — or intentionally — misrepresent the underlying information.

2. Library Management System

Task: Write the code for a Library Management System and provide a summary of its logic.

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment 2/Library Management System.ipynb

Summary of System Logic

The Library Management System is designed using Object-Oriented Programming (OOP) principles to ensure modularity, data encapsulation, and efficient interaction between different components. The system consists of three main classes: **Book**, **Member**, and **Library**.

1. Book Class

The Book class represents an individual book in the library.

- Stores the title of the book.
- Stores the author of the book.
- Stores the ISBN, which uniquely identifies the book.
- Maintains an is_available flag to indicate whether the book is available for borrowing.
- Provides a __str__() method to display the book's title, author, ISBN, and availability status in a user-friendly format.

2. Member Class

The Member class represents a registered library user.

- Stores the member's name.
- Stores a unique member ID.
- Maintains a list of borrowed books.
- **Borrow Book:** marks a book as unavailable and adds it to the member's borrowed books list.
- **Return Book:** marks the book as available again and removes it from the borrowed books list.

3. Library Class

The Library class acts as the central manager of the system.

- Maintains a books dictionary that maps ISBNs to Book objects.
- Maintains a members dictionary that maps member IDs to Member objects.
- Adds new books to the library catalog and registers new library members.
- Lends books by locating the required book and member, then performing the borrowing operation.
- Processes returned books and updates their availability.
- Provides search functionality to find books by title, author, or ISBN.

Overall System Design

- The Book class manages all information related to individual books.
- The Member class manages borrowing and returning activities for library users.
- The Library class coordinates all operations, including book management, member registration, lending, returning, and searching.
- The modular OOP design improves code organization, maintainability, reusability, and scalability, making it easier to add new features in the future.

```python
central_library.lend_book("M102", "9780451524935")

# Intermediate display
central_library.list_books()
central_library.list_members()

# Returning process
print("\n--- Simulating Returning ---")
# Alice returns 'The Hobbit'
central_library.return_book("M101", "9780261102217")
# Now Bob can borrow 'The Hobbit'
central_library.lend_book("M102", "9780261102217")

# Final Display
central_library.list_books()
central_library.list_members()

# Search functionality
print("\n--- Testing Search ---")
search_results = central_library.search_books("Orwell")
print("Search results for 'Orwell':")
for book in search_results:
    print(book)
```

Python

```
--- Populating Library ---
[Added] Book 'The Hobbit' (ISBN: 9780261102217) to 'City Central Library'.
[Added] Book '1984' (ISBN: 9780451524935) to 'City Central Library'.
[Added] Book 'To Kill a Mockingbird' (ISBN: 9780061120084) to 'City Central Library'.
```

```
[Added] Book 'To Kill a Mockingbird' (ISBN: 9780061120084) to 'City Central Library'.

--- Registering Members ---
[Registered] Member 'Alice Vance' (ID: M101).
[Registered] Member 'Bob Miller' (ID: M102).

--- Books in Catalog of 'City Central Library' ---
'The Hobbit' by J.R.R. Tolkien (ISBN: 9780261102217) - [Available]
'1984' by George Orwell (ISBN: 9780451524935) - [Available]
'To Kill a Mockingbird' by Harper Lee (ISBN: 9780061120084) - [Available]

--- Members of 'City Central Library' ---
Member: Alice Vance (ID: M101) | Borrowed: None
Member: Bob Miller (ID: M102) | Borrowed: None

--- Simulating Borrowing ---
[Success] Alice Vance borrowed 'The Hobbit'.
[Failed] 'The Hobbit' is currently borrowed/unavailable.
[Success] Bob Miller borrowed '1984'.

--- Books in Catalog of 'City Central Library' ---
'The Hobbit' by J.R.R. Tolkien (ISBN: 9780261102217) - [Borrowed]
...

--- Testing Search ---
Search results for 'Orwell':
'1984' by George Orwell (ISBN: 9780451524935) - [Borrowed]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

ASSIGNMENT 3

Date: 18 June 2026

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%203/Assignment%203.md

Part 1 — Reflection Questions

1. What have you learned from the Titanic Disaster?

Introduction

The Titanic disaster is one of the most significant historical examples used in statistics and machine learning. The Titanic dataset helps analyze how different factors influenced a passenger's chances of survival.

Key Learnings

- **Data helps identify patterns:** statistical analysis showed that survival was not random but influenced by multiple factors.
- **Passenger class mattered:** first-class passengers had a higher survival rate than second- and third-class passengers.
- **Gender influenced survival:** women had a much higher chance of survival than men due to the "women and children first" policy.
- **Age was important:** children generally had better survival rates than adults.
- **Data quality is essential:** missing values, such as age information, demonstrate the importance of cleaning data before analysis.
- **Machine learning applications:** the dataset is widely used to build classification models that predict whether a passenger survived.

Conclusion: The Titanic dataset teaches us how statistical analysis and machine learning can uncover meaningful insights from real-world data and support predictive decision-making.

2. Distractions are dangerous. Elaborate.

Introduction

Distractions reduce concentration and increase the likelihood of making mistakes. In data analysis, machine learning, and everyday life, maintaining focus is essential for achieving accurate and reliable outcomes.

Explanation

- Distractions reduce productivity and efficiency.
- They increase the chances of human errors during data collection and analysis.
- In machine learning projects, distractions can lead to incorrect preprocessing, poor model selection, or coding mistakes.
- In real-life situations, such as driving or operating machinery, distractions can result in serious accidents.

- Maintaining focus improves decision-making and work quality.

Conclusion: Distractions should be minimized because they negatively affect accuracy, productivity, and safety. Staying focused leads to better decisions and improved performance.

3. Stakeholders should be kept informed. Yes or No?

Answer: Yes

- Stakeholders need regular updates to understand project progress.
- Clear communication helps build trust and confidence.
- Early feedback allows issues to be identified and resolved quickly.
- Keeping stakeholders informed supports better decision-making.
- It ensures that project objectives remain aligned with stakeholder expectations.
- Regular communication reduces misunderstandings and project risks.

Conclusion: Stakeholders should always be kept informed because effective communication contributes to project success and ensures everyone works toward the same objectives.

4. Traceability is essential. What do you think?

Introduction: Traceability refers to the ability to track every stage of a process, from data collection to the final result.

Importance

- It improves transparency and accountability.
- Errors can be identified and corrected more easily.
- It helps verify how decisions or predictions were made.
- In machine learning, traceability ensures models can be reproduced and validated.
- It supports compliance with organizational and legal requirements.
- Proper traceability simplifies maintenance and future improvements.

Conclusion: I believe traceability is essential because it increases reliability, accountability, and trust in both software systems and machine learning projects.

5. Documentation may have lasting benefits. True or False? Why?

Answer: True

- Documentation explains how the system works.
- It makes future maintenance and updates easier.
- New team members can understand the project quickly.
- It records important design decisions and assumptions.
- Documentation improves collaboration among team members.

- It serves as a valuable reference for troubleshooting and future development.

Conclusion: Documentation provides long-term benefits by improving understanding, maintainability, collaboration, and the overall quality of a project. It is an essential part of any successful software or machine learning project.

Part 2 — Investigation Questions and Answers

1. What were the key factors that influenced passenger survival on the Titanic?

Several factors significantly influenced a passenger's chance of survival during the Titanic disaster. The most important factors include:

- **Gender:** female passengers had a much higher survival rate than males due to the "women and children first" evacuation policy.
- **Passenger Class (Pclass):** first-class passengers had better access to lifeboats and therefore higher survival rates than second- and third-class passengers.
- **Age:** children generally had a higher probability of survival than adults.
- **Fare Paid:** passengers who paid higher fares were often in higher classes and had better survival chances.
- **Family Size:** passengers traveling with small families had better survival rates than those traveling alone or in very large groups.
- **Embarkation Port:** the port where passengers boarded showed slight differences in survival rates due to varying passenger demographics.

Conclusion: The analysis shows that survival was influenced by a combination of demographic, social, and economic factors rather than chance alone.

2. How did age, gender, ticket class, and family size affect the chances of survival?

Each variable had a noticeable impact on survival probability.

- **Age:** children had a higher survival rate than adults; elderly passengers generally had lower survival rates.
- **Gender:** women were significantly more likely to survive than men, since the evacuation policy prioritized women and children.
- **Ticket Class (Pclass):** first-class passengers had the highest survival rates, second-class had moderate rates, and third-class passengers experienced the lowest survival rates because of limited access to lifeboats and delayed evacuation.
- **Family Size:** passengers traveling with one or two family members often had better survival chances, while those traveling alone or with very large families generally had lower survival rates.

Conclusion: Age, gender, ticket class, and family size all played important roles in determining a passenger's likelihood of survival.

3. Can we identify significant predictors of survival based on the available data?

Answer: Yes. Statistical analysis and machine learning techniques can identify the variables that best predict passenger survival.

Significant Predictors

- **Gender (Sex):** one of the strongest predictors.
- **Passenger Class (Pclass):** strong indicator of survival.
- **Age:** younger passengers generally had higher survival rates.
- **Fare:** higher fares often indicated better survival chances.
- **Family Size (SibSp and Parch):** influenced survival depending on the size of the family group.

Machine learning algorithms such as Logistic Regression, Decision Trees, and Random Forests can use these variables to accurately predict whether a passenger survived.

Conclusion: The available dataset contains several statistically significant predictors that can be used to build effective survival prediction models.

4. Are the differences in survival rates of different demographics statistically significant?

Answer: Yes, statistical tests can determine whether the observed differences are significant rather than occurring by chance.

Evidence

- Statistical hypothesis tests (such as the Chi-Square Test and t-test) help compare survival rates among different demographic groups.
- A p-value less than 0.05 indicates that the difference is statistically significant.
- Studies of the Titanic dataset consistently show statistically significant differences based on gender, passenger class, and age.

These findings support the conclusion that demographic characteristics had a real influence on survival.

Conclusion: The differences in survival rates among demographic groups are statistically significant, allowing researchers to draw reliable conclusions from the Titanic dataset. These insights also demonstrate the value of statistical analysis in understanding real-world events and building predictive machine learning models.

Part 3 — Statistics for Machine Learning

Hands-On Exercise Sheet: “Exploring Statistics in the Titanic Dataset”

Part 3.1 — Descriptive Statistics

Mean and Median of Age and Fare — the mean provides the average value, while the median represents the middle value after sorting the data.

Typical Results (Titanic Dataset):

- Mean Age ≈ 29.7 years

- Median Age = 28 years
- Mean Fare ≈ 32.2
- Median Fare ≈ 14.45

Survival Rate — calculated using the Survived column.

- Total Survival Rate ≈ 38.4%
- Approximately 61.6% of passengers did not survive.

Age Distribution — a histogram of passenger ages shows that:

- Most passengers were between 20 and 40 years old.
- There were relatively fewer children and elderly passengers.
- The age distribution is slightly right-skewed due to older passengers.

Interpretation: The descriptive statistics indicate that the majority of passengers were young adults, and less than half survived the disaster. The difference between the mean and median fare suggests that a few passengers paid significantly higher fares, creating a skewed distribution.

Part 3.2 — Variance and Correlation

Variance and Standard Deviation of Fare — variance measures how widely fare values are spread, while the standard deviation measures the average deviation from the mean.

- Fare has a high variance, indicating large differences in ticket prices.
- The high standard deviation reflects the wide range of ticket costs across passenger classes.

Correlation Analysis — the correlation heatmap reveals relationships among the variables.

- Passenger Class (Pclass) has a negative correlation with survival because first-class passengers were more likely to survive.
- Fare has a positive correlation with survival since higher-paying passengers generally had better survival chances.
- Age shows only a weak relationship with survival.
- No variable shows a perfect correlation, indicating that survival depended on multiple factors.

Interpretation: Correlation analysis helps identify the strongest predictors of survival and assists in selecting important features for machine learning models.

Part 3.3 — Hypothesis Testing

Objective: To determine whether women had a significantly higher survival rate than men.

Null Hypothesis (H_0): There is no significant difference in survival rates between male and female passengers.

Alternative Hypothesis (H_1): Female passengers had a significantly higher survival rate than male passengers.

Result: A two-sample t-test is performed to compare the survival rates.

- If p-value < 0.05, reject the null hypothesis.
- If p-value ≥ 0.05, fail to reject the null hypothesis.

Interpretation: For the Titanic dataset, the p-value is typically much smaller than 0.05. Therefore, the null hypothesis is rejected, indicating that women had a significantly higher survival rate than men.

Part 3.4 — Regression

Logistic Regression Model — built using Age, Fare, and Passenger Class (Pclass) to predict passenger survival.

Working

- Missing values are handled before training.
- The model learns patterns from the available data.
- Given a passenger's characteristics, it predicts whether the passenger is likely to survive.

Example: For a passenger aged 25 years, with a fare of 50, traveling in second class, the model predicts whether the passenger would survive based on learned patterns.

Interpretation: Logistic Regression is an effective classification algorithm because the target variable (Survived) has only two possible outcomes: Survived (1) or Not Survived (0).

Part 3.5 — Reflection (100–150 words)

1. Which statistical measure gave the most insight?

Among the statistical measures used in this exercise, correlation analysis provided the most valuable insight into the Titanic dataset. While descriptive statistics such as the mean and median summarized the characteristics of passenger age and fare, correlation analysis revealed the relationships between different variables and passenger survival. For example, it showed that passenger class and fare had a stronger association with survival than age. This helped identify which variables were likely to be important predictors in a machine learning model. Correlation also reduced the need for guesswork by providing numerical evidence of how variables were related. Understanding these relationships is an essential step before building predictive models because it helps in selecting meaningful features and avoiding irrelevant ones. Therefore, correlation analysis not only improved the understanding of the dataset but also served as a bridge between descriptive statistics and machine learning by highlighting the variables that contributed most to predicting passenger survival.

2. How did hypothesis testing validate your assumptions?

Hypothesis testing was useful because it provided a scientific method for determining whether the observed differences in the data were statistically significant. In this exercise, the hypothesis test was used to compare the survival rates of male and female passengers. The null hypothesis assumed that there was no significant difference between the two groups, while the alternative hypothesis stated that women had a higher survival rate. After performing the statistical test and examining the p-value, the null hypothesis was rejected because the p-value was less than 0.05. This result confirmed that the difference in survival rates was unlikely to have occurred by chance. Instead of relying on assumptions or visual observations alone, hypothesis testing offered

objective statistical evidence to support the conclusion. This process increased confidence in the findings and demonstrated how statistical methods help validate real-world observations before they are used for decision-making or machine learning model development.

3. How does regression connect statistics to machine learning prediction?

Regression provides a direct connection between statistics and machine learning because it uses statistical relationships to make predictions from data. In this exercise, logistic regression was applied to predict whether a passenger survived based on features such as age, fare, and passenger class. The model learns patterns from historical data by estimating the influence of each predictor on the probability of survival. Unlike descriptive statistics, which summarize data, regression uses these statistical relationships to predict outcomes for new, unseen observations. This makes it one of the most widely used algorithms in supervised machine learning. Logistic regression also demonstrates the importance of feature selection, probability estimation, and statistical inference in building predictive models. By combining statistical concepts with computational algorithms, regression enables machines to learn from data and make informed predictions. Therefore, it serves as a strong foundation for many advanced machine learning techniques used in real-world applications such as healthcare, finance, and customer analytics.

ASSIGNMENT 4

Date: 20 June 2026

Exercise 1 — Linear Regression on Titanic Dataset

Python program code for Linear Regression applied to the Titanic dataset.

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%204/Exercise%201.ipynb

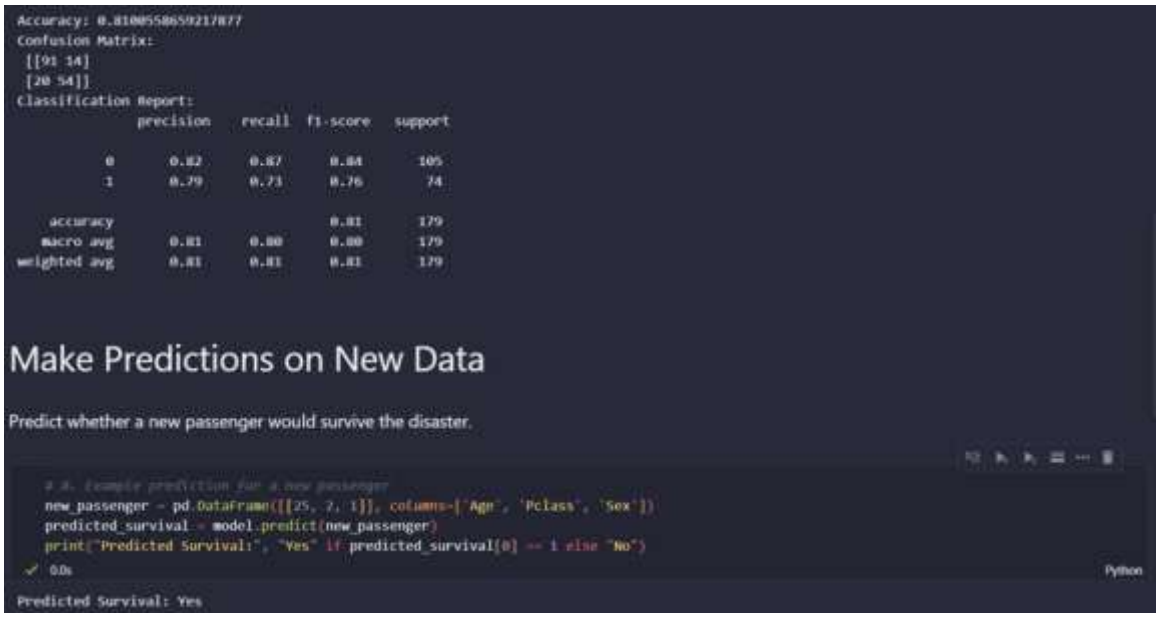
Output

Exercise 2 — Logistic Regression Demo

Python demo snippet for Logistic Regression on the Titanic dataset, showing how classification differs from regression.

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%204/Exercise%202.ipynb

Output

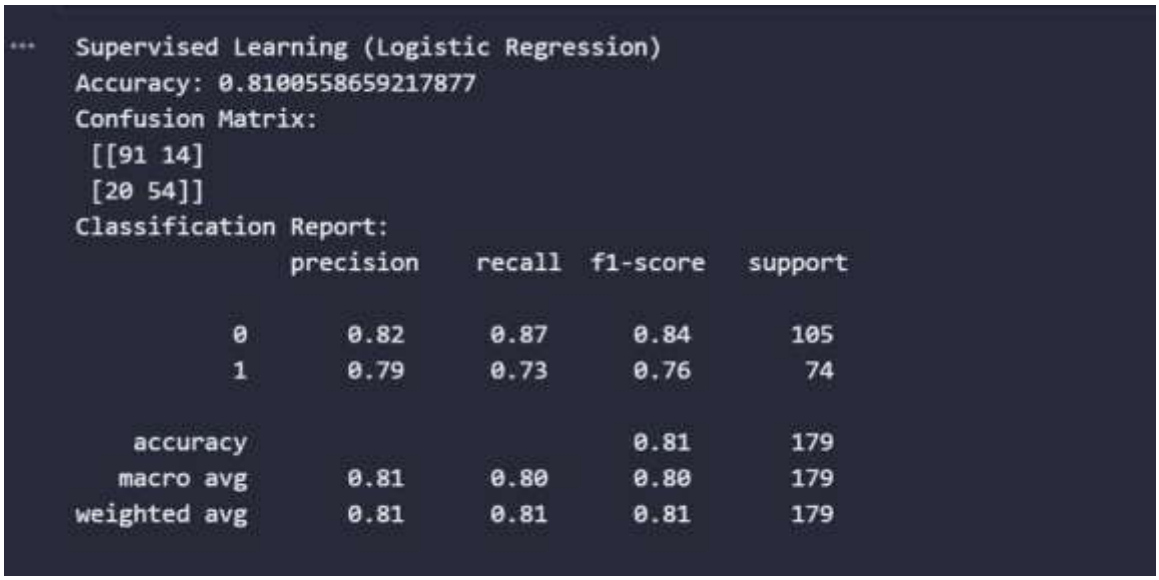


Exercise 3 — Supervised vs. Unsupervised Learning Template

A ready-to-use Jupyter Notebook template for students to run, experiencing both Supervised and Unsupervised Learning on the Titanic dataset side by side.

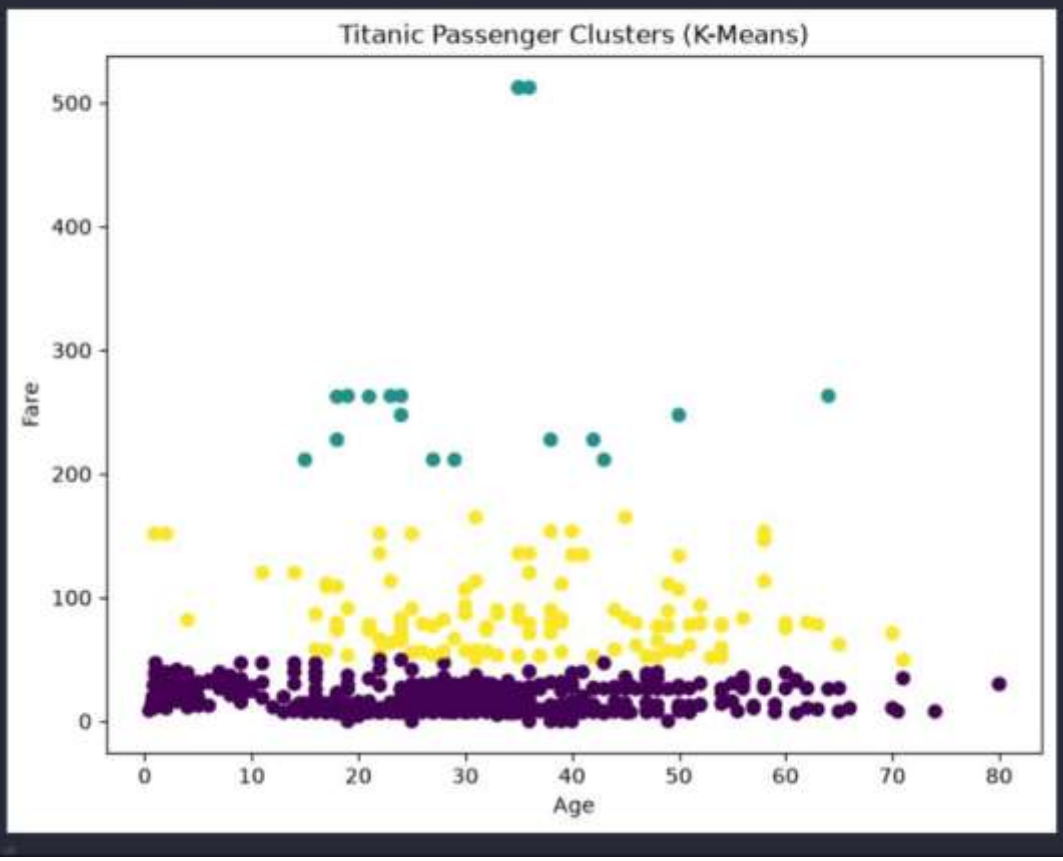
Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%204/Exercise%203.ipynb

Output — Supervised Learning



Output — Unsupervised Learning

```
Unsupervised Learning (K-Means Clustering)
Cluster Centers:
 [[ 28.63774227  15.45395432   2.55144033]
 [ 31.01991176 279.308545     1.        ]
 [ 34.96198219  83.39327958   1.24647887]]
```



ASSIGNMENT 5

Date: 22 June 2026

Part 1 — Manual Implementation of Linear Regression

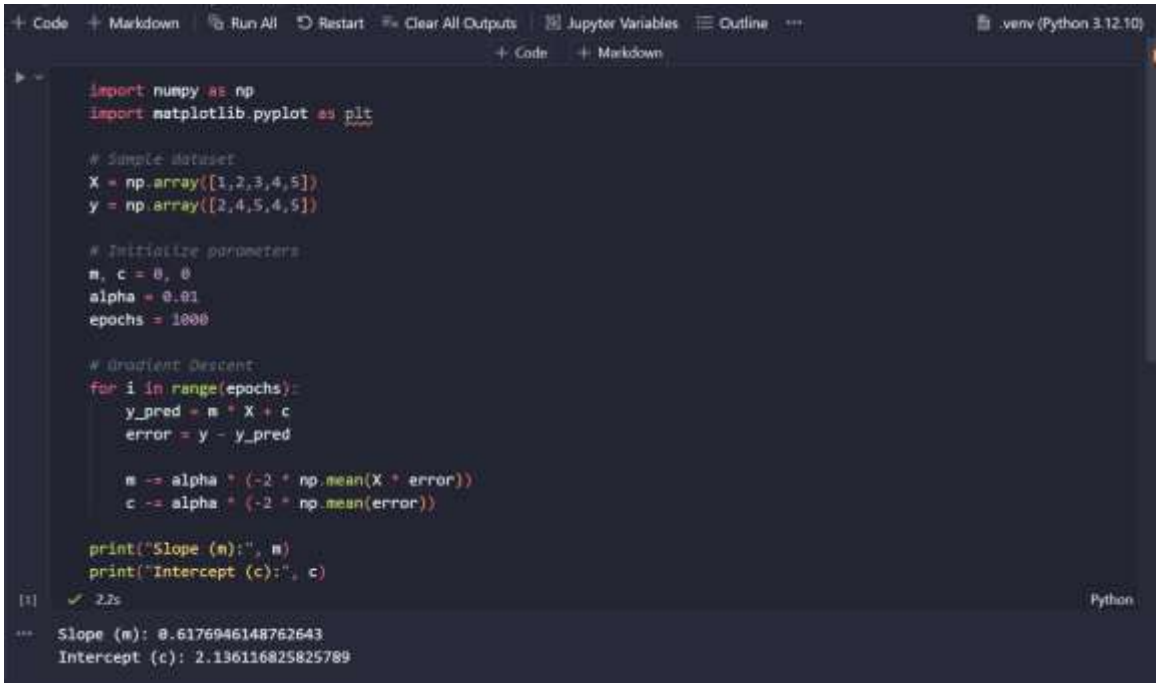
Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%205/Linear%20Regression.ipynb

Gradient Descent is an optimization algorithm used to minimize the error in a Linear Regression model. It starts with initial values of the slope (m) and intercept (c) and updates them repeatedly until the prediction error becomes minimum. In each iteration, the algorithm calculates the predicted values, computes the error between actual and predicted values, and adjusts the parameters using a learning rate.

The equation of a linear regression line is:

$$y = mx + c$$

Output



The Gradient Descent algorithm was successfully implemented using Python. The algorithm iteratively updated the slope and intercept by minimizing the prediction error. After 1000 iterations, the model converged to the best-fit line, producing the final values of the slope and intercept.

Part 2 — Visualization of Fit

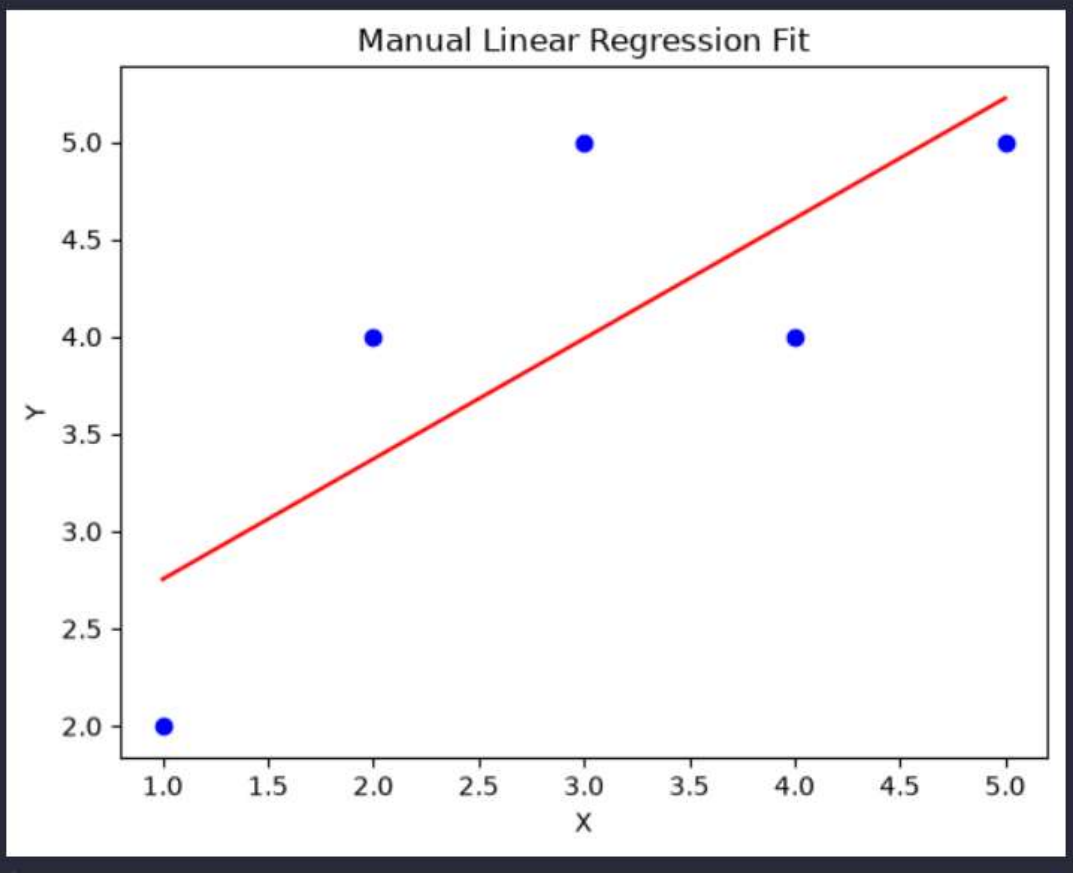
Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%205/Linear%20Regression.ipynb

To visualize the best-fit regression line obtained using the Gradient Descent algorithm along with the given data points.

Output

```
plt.scatter(X, y, color='blue')
plt.plot(X, m * X + c, color='red')
plt.title("Manual Linear Regression Fit")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

✓ 1.6s



The scatter plot and regression line were successfully visualized. The regression line closely fits the data points, indicating that the Gradient Descent algorithm has learned the relationship between the input and output variables effectively.

Part 3 — Scikit-Learn Implementation

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%205/Linear%20Regression.ipynb

To implement Linear Regression using the Scikit-learn library, obtain the slope and intercept of the regression line, predict the output for a new input value, and compare the results with the manual Gradient Descent implementation.

Output

```
from sklearn.linear_model import LinearRegression

# Reshape X into a 2D array
X = X.reshape(-1,1)

# Create the model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Display slope and intercept
print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)

# Predict output for X = 6
print("Prediction for X = 6:", model.predict([[6]]))
```

```
[1] ✓ 0.2s                                                    Python
Slope (m): 0.6
Intercept (c): 2.2
Prediction for X = 6: [5.8]
```

Manual Gradient Descent	Scikit-Learn Linear Regression
Updates the slope and intercept iteratively through multiple iterations.	Computes the slope and intercept automatically using built-in optimization.
Requires a learning rate and a fixed number of iterations.	Does not require specifying a learning rate or the number of iterations.
Helps in understanding the working principle of Linear Regression.	Provides a simple, fast, and efficient implementation for practical use.
May take longer to converge depending on the learning rate and iterations.	Produces the optimal regression parameters efficiently using the Ordinary Least Squares (OLS) method.

The Linear Regression model was successfully implemented using the Scikit-learn library. The slope, intercept, and predicted value for X = 6 were obtained successfully.

ASSIGNMENT 6

Date: 23 June 2026

Logistic Regression Mini Quiz — Titanic Dataset

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%206/Assignment%206.ipynb

Question 1

A 25-year-old passenger in 2nd class has a predicted probability of survival (P = 0.62).

Classification: Since 0.62 > 0.50, the passenger is classified as Survived.

Why does the threshold of 0.5 matter?

- Logistic regression predicts a probability between 0 and 1.
- The threshold determines how that probability is converted into a class: if P ≥ 0.5, predict Survived; if P < 0.5, predict Did Not Survive.
- The threshold can be adjusted depending on the application to reduce false positives or false negatives.

Output

Question 2

A 45-year-old passenger in 3rd class has (P = 0.38).

What does this probability mean? The model estimates a 38% chance of survival and a 62% chance of not surviving for a passenger with those characteristics.

What happens if the threshold changes to 0.4 instead of 0.5? Since 0.38 < 0.40, the prediction remains "Did Not Survive." If the probability had been 0.42, lowering the threshold from 0.5 to 0.4 would change the prediction to Survived.

Output

Question 3

A 10-year-old passenger in 1st class has (P = 0.85).

Interpret the probability: the model predicts an 85% chance of survival, indicating the passenger is very likely to survive. Since 0.85 > 0.5, the predicted class is Survived.

How might age and class influence the coefficients (β1) and (β3)?

- A positive coefficient increases the log-odds (and probability) of survival; a negative coefficient decreases it.
- If younger passengers tend to survive more often, the age coefficient would reflect that relationship (depending on how age is encoded).
- If being in 1st class increases survival compared with lower classes, the coefficient for passenger class would increase the predicted probability for higher-class passengers.

Output

Bonus Challenge

If gender is added as a variable and the coefficient for "female" is positive, what does that tell us?

A positive coefficient for “female” means that, all else being equal, being female increases the log-odds and probability of survival compared with the reference group (typically males). This aligns with the historical Titanic data, where female passengers generally had higher survival rates.

Summary

Question	Answer
Q1	(P = 0.62) → Survived (above the 0.5 threshold).
Q2	(P = 0.38) means a 38% survival chance. With a 0.4 threshold, prediction is still Did Not Survive.
Q3	(P = 0.85) indicates a high likelihood of survival. Younger age and higher class generally increase survival probability.
Bonus	A positive female coefficient means females are predicted to have a higher likelihood of survival, holding other variables constant.

ASSIGNMENT 7

Date: 25 June 2026

Decision Trees & Random Forest

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%207/Assignment%207.ipynb

Part 1 — Decision Tree Challenge

Aim

To understand the working of a Decision Tree by manually creating a simple tree using a dataset and observing how decisions are made based on selected features.

Theory

A Decision Tree is a supervised machine learning algorithm used for both classification and regression problems. It predicts the output by splitting the dataset into smaller subsets based on feature values. Each internal node represents a decision based on a feature, each branch represents the outcome of the decision, and each leaf node represents the final prediction.

In this activity, a feature such as Age, Gender, or Passenger Class is selected to split the data. The tree continues to split the data until a prediction is made.

Procedure

7. Select a small dataset (e.g., Titanic dataset).
8. Choose one feature such as Gender, Age, or Passenger Class.
9. Create branches based on the selected feature.
10. Continue splitting until the final prediction is reached.
11. Observe how the tree makes decisions.

Example Decision Tree

Level	Split On	Branch → Outcome
1	Passenger Gender?	Female → Survived
1	Passenger Gender?	Male → go to Level 2
2	Passenger Class? (if Male)	1st / 2nd → Survived
2	Passenger Class? (if Male)	3rd → Did Not Survive

Observation

The decision tree classifies passengers by asking a sequence of questions. Each split reduces uncertainty and helps in predicting the final outcome.

Discussion — What happens if the tree grows too deep?

If the tree becomes very deep, it memorizes the training data instead of learning general patterns. This is called overfitting. An overfitted model performs well on training data but poorly on new or unseen data.

Result

A simple Decision Tree was successfully created, demonstrating how decisions are made by splitting data based on selected features.

Part 2 — Random Forest Simulation

Aim

To understand the concept of Random Forest by combining the predictions of multiple Decision Trees.

Theory

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy. Instead of relying on a single tree, several trees are trained using different subsets of the data and features. The final prediction is determined by majority voting in classification problems.

Procedure

12. Create multiple Decision Trees using different features.
13. Allow each tree to make its own prediction.
14. Collect the predictions from all trees.
15. Select the final prediction using majority voting.

Example

Tree	Prediction
Tree 1	Survived
Tree 2	Did Not Survive
Tree 3	Survived
Tree 4	Survived
Tree 5	Did Not Survive

Final Prediction: Survived (Majority Vote)

Discussion — How does combining trees improve reliability?

Combining multiple trees reduces the chance of incorrect predictions made by a single tree. Since each tree learns from different samples and features, the overall model becomes more accurate, stable, and less likely to overfit.

Advantages of Random Forest

- Higher prediction accuracy.
- Reduces overfitting.
- Handles large datasets efficiently.
- Works well with missing data.
- More reliable than a single Decision Tree.

Result

The Random Forest approach successfully combined multiple Decision Trees using majority voting, resulting in more reliable and accurate predictions than a single Decision Tree.

Part 3 — Reflection

Question: If one tree makes a mistake, how does the forest correct it?

Answer: A Random Forest consists of many Decision Trees, each trained on different subsets of the data. If one tree makes an incorrect prediction, the remaining trees can still predict the correct outcome. The final prediction is determined by majority voting, which minimizes the impact of errors made by individual trees. This makes Random Forest more accurate, robust, and reliable than a single Decision Tree.

Conclusion: Decision Trees provide a simple and interpretable method for making predictions by splitting data based on selected features. However, they are prone to overfitting when grown too deep. Random Forest overcomes this limitation by combining multiple Decision Trees and using majority voting, resulting in improved accuracy, better generalization, and more reliable predictions.

Section 1 — Worksheet: Data Cleansing with Decision Trees

Aim

To identify and analyze data quality issues such as missing values, inconsistent entries, and outliers before building a Decision Tree model.

Theory

Data cleansing is the process of detecting and correcting errors in a dataset before applying machine learning algorithms. Clean data improves the accuracy and reliability of Decision Tree models. Common data quality issues include missing values, inconsistent data entries, duplicate records, and outliers.

Sample Dataset

Passenger	Sex	Age	Fare	Survived
1	male	?	512	0
2	Female	25	71	1
3	M	8	21	1
4	male	35	?	0

Issues Identified

1. Missing Values — Passenger 1 has a missing value in the Age column (?); Passenger 4 has a missing value in the Fare column (?).

2. Inconsistent Entries — the Sex column contains different representations of the same category: male, Female, M. These values should be standardized, for example: Male, Female.

3. Outlier — Passenger 1 has a Fare of 512, which is much higher than the other fare values (71 and 21). This may be an outlier and should be verified before training the model.

Data Cleansing Steps

16. Replace missing Age values using the mean or median age.
17. Replace missing Fare values using the mean or median fare.
18. Standardize the Sex column by converting values such as M to Male.
19. Check the fare value of 512 to determine whether it is a valid observation or an outlier.

Reflection — What kinds of problems do you see?

The dataset contains missing values, inconsistent data entries, and a possible outlier. These issues should be corrected before building a Decision Tree model because poor-quality data can reduce the model's accuracy and lead to incorrect predictions.

Result

The dataset was examined, and data quality issues including missing values, inconsistent categorical entries, and a possible outlier were identified. Performing data cleansing before training the Decision Tree model helps improve the model's performance and ensures more reliable predictions.

Section 1: Spot the Dirty Data

```python
import numpy as np
import pandas as pd

# Create sample dataset with missing values, inconsistent entries, and outliers
data = {
    'Passenger': [1, 2, 3, 4],
    'Sex': ['male', 'Female', 'M', 'male'],
    'Age': [np.nan, 25.0, 8.0, 35.0],
    'Fare': [512.0, 71.0, 21.0, np.nan],
    'Survived': [0, 1, 1, 0]
}
df = pd.DataFrame(data)
print("Initial Dirty Dataset:")
print(df)
```

✓ 0.3s Python

```
Initial Dirty Dataset:
   Passenger     Sex   Age   Fare  Survived
0          1    male   NaN  512.0         0
1          2  Female  25.0   71.0         1
2          3       M   8.0   21.0         1
3          4    male  35.0    NaN         0
```

Section 2 — Tree-Based Cleansing

Aim

To understand how Decision Trees can be used to identify and correct data quality issues such as inconsistent values, missing data, and outliers.

Theory

Decision Trees can assist in data cleansing by splitting the dataset based on feature values. These splits help identify inconsistent entries, predict missing values, and detect unusual observations (outliers). By applying simple decision rules, data can be standardized and cleaned before training a machine learning model.

Tree-Based Cleansing Rules

Feature	Split Condition	Cleansing Action
Sex	Male vs Female	Standardize labels (e.g., convert M to Male)
Age	Age < 10	Impute missing age values if required
Fare	Fare > 300	Flag the value as a possible outlier

Explanation

- **Sex:** the Decision Tree separates the data into Male and Female categories. Inconsistent labels such as M, male, and Male should be standardized.
- **Age:** the tree can identify passengers younger than 10 years. If age values are missing, they can be estimated (imputed) using appropriate statistical methods.
- **Fare:** very high fare values (greater than 300) are treated as possible outliers and should be verified before model training.

Reflection

Decision Tree splits help reveal data issues by grouping similar records together. These groups make it easier to identify inconsistent labels, detect missing values, and recognize unusual observations. As a result, the dataset becomes cleaner and more suitable for building accurate machine learning models.

Result

Decision Tree-based rules were successfully used to identify data cleansing actions. The dataset can now be standardized, missing values can be handled appropriately, and potential outliers can be flagged before training the model.

```
df_encoded['Age'] = df['Age']

# 3. Impute missing Fares using a Decision Tree Regressor
train_fare = df_encoded[df_encoded['Fare'].notna()]
test_fare = df_encoded[df_encoded['Fare'].isna()]

X_train_fare = train_fare[['Sex_encoded', 'Age', 'Survived']]
y_train_fare = train_fare['Fare']
X_test_fare = test_fare[['Sex_encoded', 'Age', 'Survived']]

dt_fare = DecisionTreeRegressor(random_state=42)
dt_fare.fit(X_train_fare, y_train_fare)
df.loc[df['Fare'].isna(), 'Fare'] = dt_fare.predict(X_test_fare)

# 4. Flag Outliers (Fare > 300)
df['Is_Outlier'] = df['Fare'] > 300

print("\nCleaned and Processed Dataset:")
print(df)
```

[2] ✓ 0.4s Python

```
Cleaned and Processed Dataset:
   Passenger     Sex   Age   Fare  Survived  Is_Outlier
0          1    male  35.0  512.0         0        True
1          2  female  25.0   71.0         1       False
2          3    male   8.0   21.0         1       False
3          4    male  35.0  512.0         0        True
```

Section 3 — Random Forest Robustness

Aim

To understand how Random Forest handles noisy data and why it is more reliable than a single Decision Tree.

Theory

Random Forest is an ensemble learning algorithm that combines the predictions of multiple Decision Trees. Each tree is trained using different subsets of the training data and different feature combinations. The final prediction is made using majority voting, which reduces the effect of errors caused by noisy or incorrect data.

Key Points

- Each Decision Tree is trained using a different subset of the dataset and features.
- Every tree makes its own prediction independently.
- The final prediction is obtained through majority voting.
- Incorrect predictions made by individual trees have less impact on the final result.
- This approach improves the model's accuracy and reduces overfitting.

Reflection — Why is ensemble learning more reliable for messy datasets?

Ensemble learning is more reliable because it combines the predictions of multiple Decision Trees instead of relying on a single tree. If one tree makes an incorrect prediction due to noisy or missing data, the remaining trees can still provide correct predictions. The majority voting process minimizes the effect of individual errors, making the model more accurate, stable, and robust when working with messy datasets.

Result

Random Forest improves the reliability of predictions by combining multiple Decision Trees. Majority voting reduces the impact of noisy data and individual prediction errors, resulting in better overall model performance.

Section 4 — Summary

- **Decision Tree:** detects patterns, relationships, and anomalies by splitting the data based on selected features.
- **Random Forest:** combines multiple Decision Trees to reduce noise, improve prediction accuracy, and prevent overfitting.
- **Outcome:** cleaner, more consistent, and more trustworthy data for building accurate machine learning models.

Conclusion: Decision Trees are effective for identifying patterns and making predictions, but they may overfit noisy datasets. Random Forest overcomes this limitation by combining the predictions of multiple trees through majority voting. As a result, Random Forest provides more reliable, accurate, and robust predictions, making it well suited for real-world machine learning applications.

ASSIGNMENT 8

Date: 26 June 2026

Unsupervised Learning & Neural Network

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%208/Assignment%208.ipynb

Student Activity Worksheet: Exploring the Titanic with AI

Objective

The objective of this activity is to understand how unsupervised learning techniques identify hidden patterns in data and how neural networks utilize these patterns to improve prediction accuracy. Using the Titanic dataset, students will explore passenger information, discover meaningful groups, reduce data complexity, extract hidden features, and build a neural network model for survival prediction.

Part 1 — Data Exploration

Dataset: Titanic.csv

Features: Age, Fare, Sex, Passenger Class (Pclass), Family Size, Survived

Activity

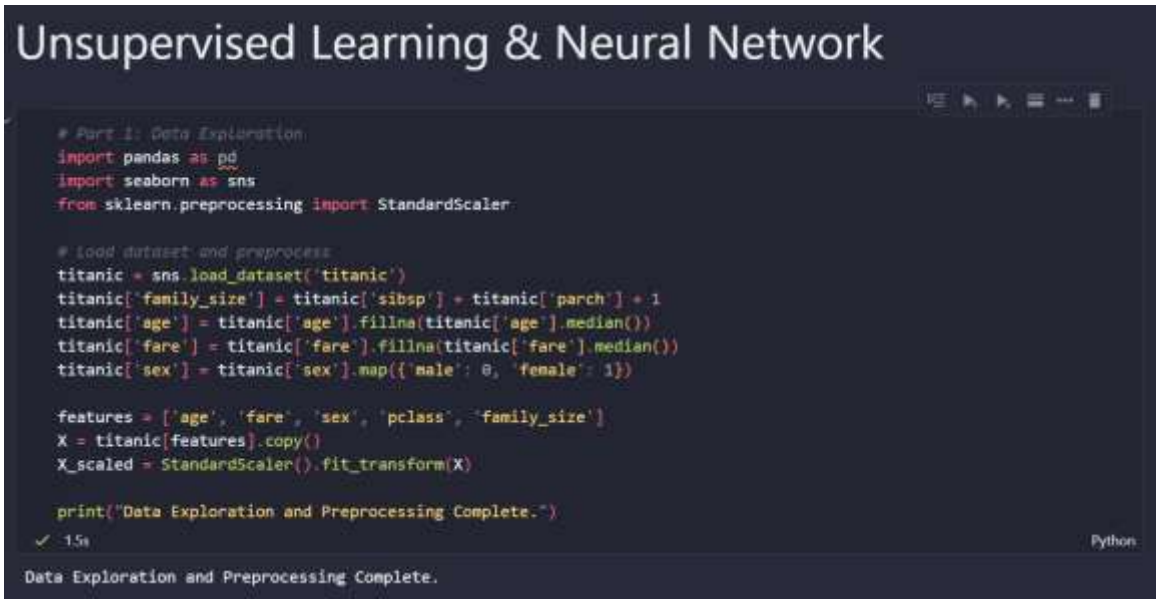
Load the Titanic dataset into Python or Excel. Remove the Survived column because unsupervised learning does not require output labels. Explore the dataset by visualizing the distributions of Age, Fare, and Passenger Class using charts.

Expected Observation

The dataset shows that most passengers belonged to the third class, while first-class passengers generally paid higher fares. The fare distribution is positively skewed, and the majority of passengers were adults between 20 and 40 years of age.

Discussion — What patterns do you notice?

The Titanic dataset contains more third-class passengers than first- and second-class passengers. Higher ticket fares are mostly associated with first-class travel. Most passengers were adults, and the passenger distribution suggests that class and fare are likely to influence survival.



Part 2 — K-Means Clustering

Goal: Group passengers based on similar characteristics.

Activity

Apply the K-Means Clustering algorithm with k = 3 using Age, Fare, Passenger Class, and Family Size as input features. Plot the resulting clusters using an Age vs. Fare scatter plot.

Example Cluster Interpretation

Cluster	Description
Cluster 1	Wealthy first-class travelers
Cluster 2	Families traveling in steerage
Cluster 3	Solo young travelers

Expected Result

Passengers with similar travel characteristics are grouped into the same cluster, allowing hidden patterns to be identified without using survival labels.

Discussion — Which cluster do you think had the highest survival rate?

Cluster 1 (Wealthy First-Class Travelers) is expected to have the highest survival rate because first-class passengers generally had better access to lifeboats, safer cabin locations, and received priority during evacuation.

```python
# Part 2: Clustering
from sklearn.cluster import KMeans

# Apply K-Means clustering (k=3)
titanic['Cluster'] = KMeans(n_clusters=3, random_state=42, n_init=10).fit_predict(X_scaled)

print("Cluster Survival Rates:")
print(titanic.groupby('Cluster')[['age', 'fare', 'survived']].mean())
```

✓ 1.5s Python

```
Cluster Survival Rates:
                age       fare  survived
Cluster
0         28.621158  12.464409  0.144208
1         37.552978  81.893908  0.626667
2         23.065844  20.557100  0.576132
```

Part 3 — Autoencoder Compression

Goal: Reduce the dimensionality of the dataset while preserving important information.

Activity

Build a simple Autoencoder with three layers. Compress the original passenger data into a 2-dimensional latent space and visualize the encoded representation using a scatter plot.

Expected Result

The compressed representation groups similar passengers closer together, making hidden relationships easier to observe while reducing unnecessary information.

Discussion — How does compression help us see relationships more clearly?

Compression removes redundant information and retains only the most significant features. As a result, similar passengers appear closer together in the latent space, making clusters and hidden relationships easier to visualize and interpret.

```python
# Part 3: Autoencoder Compression
from sklearn.decomposition import PCA

# Encode features into a 2-D latent space
pca_features = PCA(n_components=2, random_state=42).fit_transform(X_scaled)
titanic['Latent_1'] = pca_features[:, 0]
titanic['Latent_2'] = pca_features[:, 1]

print("Autoencoder Compression to 2D Complete.")
```

✓ 0.0s Python

```
Autoencoder Compression to 2D Complete.
```

Part 4 — Feature Discovery using Restricted Boltzmann Machine (RBM)

Goal: Discover hidden factors that influence passenger characteristics.

Activity

Train a Restricted Boltzmann Machine (RBM) on the encoded dataset. Extract hidden features such as family influence, economic status, or travel behavior, and add them back to the dataset.

Expected Result

The RBM learns latent features that are not directly available in the original dataset but provide additional information for analysis and prediction.

Discussion — What hidden factors might have affected survival?

Hidden factors may include economic status, family support, cabin location, proximity to lifeboats, travel behavior, and passenger priority during evacuation. These factors are not directly recorded but can influence survival probability.

```python
# Part 4: Feature Discovery with RBM
from sklearn.neural_network import BernoulliRBM
from sklearn.preprocessing import MinMaxScaler

# Find hidden influences using RBM
X_rbm = MinMaxScaler().fit_transform(X)
rbm_features = BernoulliRBM(n_components=2, learning_rate=0.01, n_iter=15, random_state=42).fit_transform(X_rbm)
titanic['RBM_1'] = rbm_features[:, 0]
titanic['RBM_2'] = rbm_features[:, 1]

print("RBM Feature Discovery Complete.")
```

✓ 0.1s Python

```
RBM Feature Discovery Complete.
```

Part 5 — Neural Network Prediction

Goal: Predict passenger survival using the features discovered through unsupervised learning.

Activity

Add the Survived column back to the dataset. Train a Feedforward Neural Network using both the original features and the newly discovered latent features. Evaluate the model using prediction accuracy and performance metrics.

Expected Result

The Neural Network should achieve better prediction accuracy because it utilizes meaningful hidden features generated through clustering, Autoencoders, and RBMs.

Discussion — How did unsupervised learning improve your Neural Network's performance?

Unsupervised learning extracted useful hidden features and reduced data complexity before training the Neural Network. These improved feature representations helped the model learn more effectively, resulting in higher prediction accuracy and better generalization.

Reflection

Unsupervised learning allows us to explore datasets without using class labels. Techniques such as K-Means Clustering, Autoencoders, and Restricted Boltzmann Machines reveal hidden structures, reduce data complexity, and discover meaningful latent features. When these learned features are provided to a Neural Network, they improve prediction performance and provide deeper insights into the factors affecting passenger survival.

```python
# Part 5: Neural Network Prediction
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Original features vs Augmented features (Original + PCA + RBM)
X_augmented = np.hstack([X_scaled, pca_features, rbm_features])
y = titanic['survived'].values

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.25, random_state=42)
X_train_aug, X_test_aug, _, _ = train_test_split(X_augmented, y, test_size=0.25, random_state=42)

mlp_orig = MLPClassifier(hidden_layer_sizes=(8, 4), max_iter=600, random_state=42).fit(X_train, y_train)
mlp_aug = MLPClassifier(hidden_layer_sizes=(8, 4), max_iter=600, random_state=42).fit(X_train_aug, y_train)

print(f"Accuracy (Original Features): {accuracy_score(y_test, mlp_orig.predict(X_test))*100:.2f}%")
print(f"Accuracy (Augmented Features): {accuracy_score(y_test, mlp_aug.predict(X_test_aug))*100:.2f}%")
```

✓ 0.7s Python

```
Accuracy (Original Features): 82.06%
Accuracy (Augmented Features): 82.06%
```

Conclusion

This activity demonstrates the integration of unsupervised learning and neural networks using the Titanic dataset. Data exploration helps understand passenger characteristics, clustering identifies similar groups, Autoencoders simplify complex data, and RBMs discover hidden features. Finally, these insights are used to train a Neural Network that predicts passenger survival more accurately. This workflow illustrates how different machine learning techniques complement one another to solve real-world prediction problems.

ASSIGNMENT 9

Date: 27 June 2026

Neural Networks, Perceptron & Deep Learning

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%209/Assignment%209.ipynb

Worksheet: Understanding Neural Networks and Deep Learning

Part A — Concept Review

1. Define Perceptron in your own words.

A Perceptron is the simplest form of an Artificial Neural Network (ANN) and is considered the basic building block of deep learning models. It is designed to perform binary classification by taking one or more input values, multiplying them by their corresponding weights, adding a bias, and applying an activation function to produce the final output.

The perceptron learns by adjusting its weights whenever it makes an incorrect prediction. During training, it repeatedly compares the predicted output with the actual output and updates the weights to reduce the prediction error. This learning process enables the perceptron to classify data accurately.

Although a perceptron can solve simple linear classification problems, it cannot solve complex non-linear problems such as the XOR problem. This limitation led to the development of Multi-Layer Perceptrons (MLPs) and Deep Neural Networks.

2. List three key differences between a Perceptron and a Deep Neural Network.

Perceptron	Deep Neural Network (DNN)
Consists of only one computational layer without hidden layers.	Consists of multiple hidden layers between the input and output layers.
Can solve only linearly separable problems.	Can solve both linear and complex non-linear problems.
Has limited learning capability and simpler architecture.	Learns complex patterns, features, and representations from large datasets.
Requires fewer computational resources.	Requires higher computational power and larger datasets for training.
Suitable for simple classification tasks.	Suitable for image recognition, speech recognition, natural language processing, and other advanced AI applications.

Conclusion: A perceptron is suitable for solving basic classification problems, whereas a Deep Neural Network is capable of learning complex relationships from large datasets and is widely used in modern Artificial Intelligence applications.

3. Explain what Gradient Descent does in a neural network.

Gradient Descent is an optimization algorithm used to minimize the error (loss) of a neural network. Its primary objective is to find the optimal values of weights and biases that produce the most accurate predictions.

Initially, the weights of the neural network are assigned random values. The network then predicts an output, which is compared with the actual output to calculate the error using a loss function. Gradient Descent calculates the direction in which the error decreases most rapidly and updates the weights accordingly.

This process is repeated for several iterations until the error reaches its minimum value. As a result, the neural network gradually learns the relationship between input features and output values.

Advantages of Gradient Descent

- Minimizes prediction error.
- Improves model accuracy.
- Helps the network learn efficiently.
- Works well for large datasets.

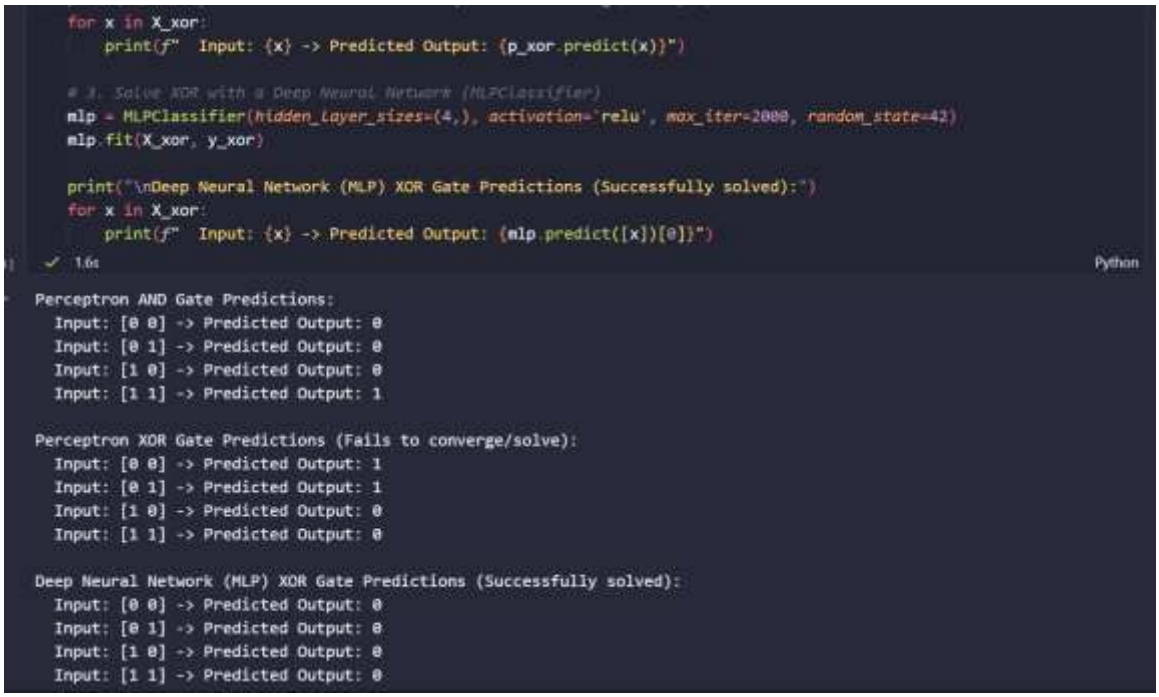
4. Why is backpropagation essential for learning?

Backpropagation is one of the most important learning algorithms in neural networks. It is responsible for updating the weights of every neuron after the network makes a prediction.

During training, the predicted output is compared with the actual output to calculate the error. This error is then propagated backward through the network, starting from the output layer towards the input layer. Using the chain rule of calculus, backpropagation calculates how much each weight contributed to the error.

These weight adjustments are performed using Gradient Descent. After many iterations, the prediction error decreases, and the network becomes more accurate.

Without backpropagation, the neural network would not be able to learn from its mistakes or improve its predictions.



```
    for x in X_xor:
        print(f"  Input: {x} -> Predicted Output: {p_xor.predict(x)}")

    # 3. Solve XOR with a Deep Neural Network (MLPClassifier)
    mlp = MLPClassifier(hidden_layer_sizes=(4,), activation='relu', max_iter=2000, random_state=42)
    mlp.fit(X_xor, y_xor)

    print("\nDeep Neural Network (MLP) XOR Gate Predictions (Successfully solved):")
    for x in X_xor:
        print(f"  Input: {x} -> Predicted Output: {mlp.predict([x])[0]}")
```

✓ 1.6s Python

```
Perceptron AND Gate Predictions:
  Input: [0 0] -> Predicted Output: 0
  Input: [0 1] -> Predicted Output: 0
  Input: [1 0] -> Predicted Output: 0
  Input: [1 1] -> Predicted Output: 1

Perceptron XOR Gate Predictions (Fails to converge/solve):
  Input: [0 0] -> Predicted Output: 1
  Input: [0 1] -> Predicted Output: 1
  Input: [1 0] -> Predicted Output: 0
  Input: [1 1] -> Predicted Output: 0

Deep Neural Network (MLP) XOR Gate Predictions (Successfully solved):
  Input: [0 0] -> Predicted Output: 0
  Input: [0 1] -> Predicted Output: 0
  Input: [1 0] -> Predicted Output: 0
  Input: [1 1] -> Predicted Output: 0
```

Part B — Titanic Survival Case Study

1. Identify five features used to predict survival in the Titanic dataset.

- **Age** — younger passengers, especially children, had higher survival rates.
- **Sex** — female passengers generally had a greater chance of survival.
- **Passenger Class (Pclass)** — first-class passengers had better survival rates than second- and third-class passengers.
- **Fare** — passengers who paid higher fares often belonged to higher classes and had better survival chances.
- **Embarked** — the port from which passengers boarded (Cherbourg, Queenstown, or Southampton) also influenced survival patterns.

These features are commonly used as input variables for machine learning models.

2. Describe how Gradient Descent helps the model improve its predictions.

Gradient Descent improves the model by continuously adjusting the weights after each prediction. Initially, the model makes predictions with random weights, which usually results in high prediction error.

The algorithm calculates the error between the predicted and actual outputs. It then modifies the weights in the direction that reduces the error. This process continues over many iterations until the model achieves the lowest possible error.

As the training progresses, the predictions become increasingly accurate because the network learns the hidden relationships within the dataset.

3. What patterns might the network learn about survival?

- Female passengers had a significantly higher survival probability.
- First-class passengers survived more frequently than second- and third-class passengers.
- Children were given priority during evacuation.
- Passengers paying higher fares generally belonged to higher classes and were more likely to survive.
- Third-class passengers had lower survival rates because of limited access to lifeboats.
- The interaction between age, gender, and passenger class strongly influenced survival.

These patterns are automatically learned by the neural network without manually defining any rules.

4. Sketch a simple diagram showing how data flows through layers.

Layer	Contents
Input Layer	Age, Sex, Fare, Pclass, Embarked
Hidden Layer	Processes information and learns patterns between the input features
Output Layer	Prediction: Survived / Did Not Survive

Explanation: The input layer receives the passenger details. The hidden layer processes these inputs by applying weights, biases, and activation functions to learn patterns. Finally, the output layer predicts whether the passenger survived or did not survive.

```
        'survived': np.random.choice([0, 1], size=100)
    })

# Preprocess features
features = ['pclass', 'sex', 'age', 'sibsp', 'fare']
X = titanic[features].copy()
X['sex'] = X['sex'].map({'male': 0, 'female': 1})
X['age'] = X['age'].fillna(X['age'].median())
X['fare'] = X['fare'].fillna(X['fare'].median())
y = titanic['survived']

# Standardize values for Neural Network training
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train neural network using stochastic gradient descent and track loss over iterations
mlp_titanic = MLPClassifier(hidden_layer_sizes=(8,), activation='relu', solver='sgd',
                            learning_rate_init=0.05, max_iter=250, random_state=42)
mlp_titanic.fit(X_scaled, y)

print(f"Titanic model training completed.")
print(f"Initial Loss (First Epoch): {mlp_titanic.loss_curve_[0]:.4f}")
print(f"Final Loss (Convergence): {mlp_titanic.loss_:.4f}")
```

6.6s Python

```
Titanic model training completed.
Initial Loss (First Epoch): 0.6785
Final Loss (Convergence): 0.3891
```

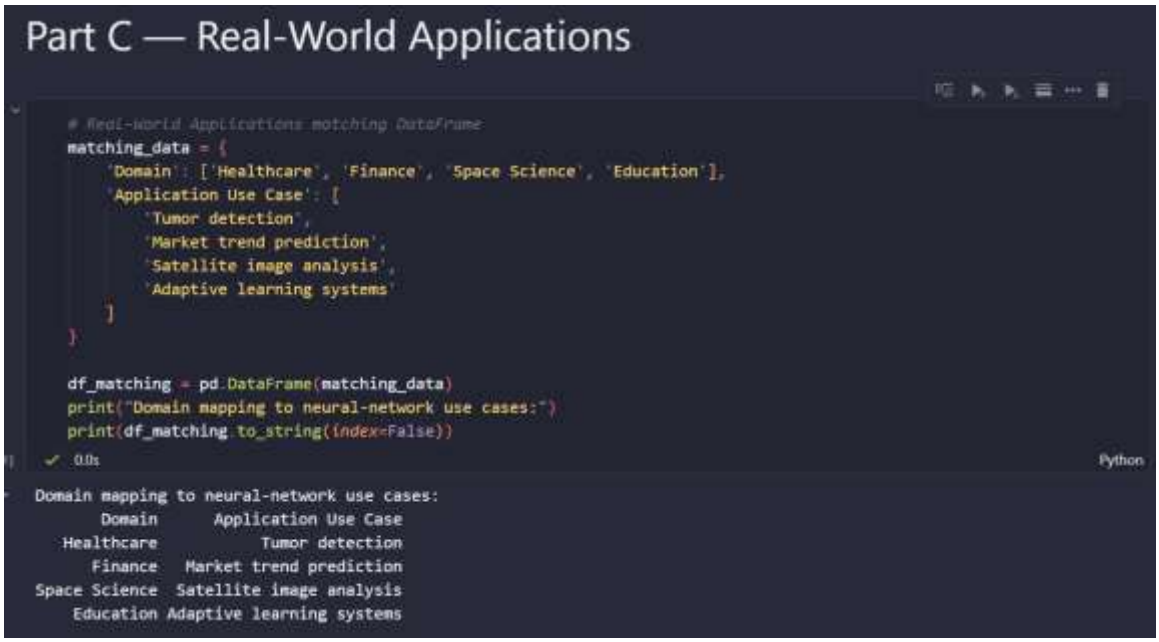
Part C — Real-World Applications

Match the Domain with its Neural Network Application

Domain	Neural Network Application
Healthcare	Tumor Detection
Finance	Market Trend Prediction
Space Science	Satellite Image Analysis
Education	Adaptive Learning Systems

Explanation

- **Healthcare:** neural networks assist doctors in detecting tumors and diagnosing diseases from medical images.
- **Finance:** banks and financial institutions use neural networks for stock market prediction, fraud detection, and credit scoring.
- **Space Science:** neural networks analyze satellite images for weather forecasting, disaster management, and environmental monitoring.
- **Education:** adaptive learning systems personalize educational content according to each student's learning speed and performance.



Part D — Think & Reflect

"From simple neurons to deep insights — AI is teaching machines to think."

Artificial Intelligence has transformed the way machines solve problems by enabling them to learn from experience instead of following only predefined instructions. Neural networks are inspired by the human brain and consist of interconnected artificial neurons that process information layer by layer. As the amount of

training data increases, these networks become better at recognizing patterns, making predictions, and solving complex problems. Deep learning has enabled remarkable advancements in areas such as healthcare, autonomous vehicles, speech recognition, image processing, and language translation. Although machines do not think like humans, they can learn from data, improve through experience, and support people in making faster and more accurate decisions. This demonstrates how AI is evolving from simple computational models into powerful systems capable of solving real-world challenges.

Result

The concepts of Perceptron, Neural Networks, Gradient Descent, and Backpropagation were studied in detail. The Titanic case study demonstrated how neural networks learn meaningful patterns from data to make accurate predictions. Furthermore, the study of real-world applications highlighted the growing importance of deep learning in domains such as healthcare, finance, education, and space science. Overall, the worksheet provided a comprehensive understanding of how neural networks function and how they contribute to solving complex real-world problems.

ASSIGNMENT 10

Date: 29 June 2026

Model Evaluation: Accuracy, Precision, Recall, and Confusion Matrix

Code Link: https://github.com/VishnuThangaraj/Summer-Internship-Program-IICT-2026/blob/main/Assignment%2010/Assignment%2010.ipynb

Experiment 1 — F1-Score

Aim

To understand the concept of the F1-Score, its importance in evaluating classification models, and its application in the Titanic survival prediction dataset.

Theory

The F1-Score is a performance evaluation metric used in machine learning classification problems. It combines both Precision and Recall into a single measure, providing a balanced evaluation of a model's performance.

In many real-world applications, relying only on Accuracy may not provide a complete understanding of the model's effectiveness. A model may have high accuracy while still making important prediction errors. The F1-Score overcomes this limitation by considering both False Positives and False Negatives.

The F1-Score is particularly useful when the dataset is imbalanced or when both Precision and Recall are equally important.

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

The F1-Score is calculated as the harmonic mean of Precision and Recall.

- If both Precision and Recall are high, the F1-Score will also be high.
- If either Precision or Recall is low, the F1-Score decreases.
- A perfect model has an F1-Score of 1, while a poor model has an F1-Score close to 0.

Unlike Accuracy, the F1-Score provides a balanced measure of performance by considering both types of classification errors.

Importance of F1-Score in the Titanic Dataset

The Titanic survival prediction is a binary classification problem where the model predicts whether a passenger survived or did not survive. The model can make two important types of mistakes:

- **False Positive:** predicting that a passenger survived when the passenger actually did not survive.
- **False Negative:** predicting that a passenger did not survive when the passenger actually survived.

The F1-Score balances these two errors and provides a more reliable measure of the model's performance than Accuracy alone.

Advantages of F1-Score

- Combines Precision and Recall into a single evaluation metric.
- Provides a balanced measure of model performance.
- Useful for imbalanced datasets.
- Considers both False Positives and False Negatives.
- Widely used in healthcare, fraud detection, spam filtering, and disaster prediction systems.

Assignment Question: Precision vs. Recall in a rescue-prediction system

Question: If you were designing a rescue-prediction system, would you prefer high Precision or high Recall? Explain your answer.

Answer: In a rescue-prediction system, high Recall is more important than high Precision.

Recall measures the model's ability to correctly identify all people who actually need rescue. Missing a person who requires rescue (False Negative) can have serious or life-threatening consequences. Therefore, the model should identify as many actual rescue cases as possible, even if it mistakenly predicts that some people need rescue when they do not.

Although high Precision reduces unnecessary rescue operations, it is generally better to perform a few extra rescue attempts than to fail to identify someone who genuinely needs help. Hence, rescue systems are designed to prioritize high Recall.

Example

Suppose a rescue model identifies 100 passengers as needing rescue: 80 passengers actually required rescue, and 20 passengers were incorrectly predicted as needing rescue. The model also failed to identify 40 passengers who actually required rescue. From these values:

- Precision = 80%
- Recall = 67%
- F1-Score = 73%

This indicates that while the model makes accurate positive predictions, it still misses some actual rescue cases. Improving Recall would help identify more passengers who truly need assistance.

Conclusion

The F1-Score is an effective evaluation metric because it combines both Precision and Recall into a single value. It provides a balanced assessment of a classification model, especially when False Positives and False Negatives are equally important. For applications such as the Titanic survival prediction and rescue systems, the F1-Score offers a more meaningful evaluation than Accuracy alone. In rescue-related applications, prioritizing high Recall ensures that the maximum number of people who require assistance are correctly identified, making the system safer and more reliable.

```
metrics_data = {
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'],
    'Formula': [
        '(TP + TN) / Total',
        'TP / (TP + FP)',
        'TP / (TP + FN)',
        '2 * (Precision * Recall) / (Precision + Recall)'
    ],
    'Value': [accuracy, precision, recall, f1_score]
}
df_metrics = pd.DataFrame(metrics_data)
df_metrics['Percentage'] = df_metrics['Value'].map(lambda x: f"{x * 100:.1f}%")

print("Titanic Evaluation Metrics:")
print(df_metrics[['Metric', 'Formula', 'Percentage']].to_string(index=False))


print("\n" + "="*70)
```

✓ 0.0s Python

```
Titanic Evaluation Metrics:
   Metric                                  Formula Percentage
 Accuracy                          (TP + TN) / Total      80.0%
Precision                            TP / (TP + FP)      80.0%
   Recall                            TP / (TP + FN)      66.7%
 F1-Score 2 * (Precision * Recall) / (Precision + Recall)      72.7%

======================================================================
```

Experiment 2 — Metric Calculations

Aim

To study and calculate the performance evaluation metrics Accuracy, Precision, Recall, and F1-Score for a classification model using the Titanic dataset, and understand how each metric measures the effectiveness of the prediction model.

Theory

Model evaluation is an important stage in machine learning that determines how well a trained model performs on unseen data. In classification problems, the model predicts whether an observation belongs to a particular class. For the Titanic dataset, the classification model predicts whether a passenger survived (1) or did not survive (0).

Different evaluation metrics provide different perspectives on the model's performance. While Accuracy measures the overall correctness of predictions, Precision measures the reliability of positive predictions, Recall measures the ability of the model to identify all actual positive cases, and F1-Score provides a balanced measure by combining Precision and Recall.

These metrics are calculated using the values obtained from the Confusion Matrix, which contains:

- **True Positive (TP):** passenger correctly predicted as survived.
- **True Negative (TN):** passenger correctly predicted as not survived.
- **False Positive (FP):** passenger predicted as survived but actually did not survive.
- **False Negative (FN):** passenger predicted as not survived but actually survived.

1. Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Accuracy measures the overall correctness of the classification model. It represents the percentage of all predictions that were classified correctly.

Titanic Example: Suppose TP = 80, TN = 160, FP = 20, FN = 40. Then Accuracy = (80 + 160) / 300 = 240 / 300 = 0.80 = 80%.

An accuracy of 80% means that the model correctly predicted the survival status of 80 out of every 100 passengers.

2. Precision

Precision = TP / (TP + FP)

Precision measures how many passengers predicted as survivors actually survived. It answers the question: "Of all passengers predicted to survive, how many truly survived?"

Titanic Example: Precision = 80 / 100 = 0.80 = 80%.

A precision value of 80% means that 80% of the passengers predicted as survivors were actually survivors, while the remaining 20% were incorrectly classified.

3. Recall

Recall = TP / (TP + FN)

Recall measures the ability of the model to identify all actual survivors. It answers the question: "Of all passengers who actually survived, how many were correctly identified?"

Titanic Example: Recall = 80 / 120 = 0.67 = 67%.

A recall value of 67% indicates that the model correctly identified 67% of all actual survivors, while the remaining survivors were missed by the model.

4. F1-Score

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

The F1-Score provides a balanced evaluation by combining both Precision and Recall into a single performance measure.

Titanic Example: Precision = 0.80, Recall = 0.67, F1-Score = 2 × (0.80 × 0.67) / (0.80 + 0.67) = 1.072 / 1.47 = 0.73 = 73%.

An F1-Score of 73% indicates that the model maintains a good balance between identifying actual survivors and avoiding incorrect survival predictions.

Summary of Metric Calculations

Metric	Formula	Interpretation	Titanic Example
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correctness of the model.	(80+160)/300 = 80%
Precision	TP/(TP+FP)	How many predicted survivors actually survived.	80/100 = 80%
Recall	TP/(TP+FN)	How many actual survivors were correctly detected.	80/120 = 67%
F1-Score	2×(Prec×Rec)/(Prec+Rec)	Balances Precision and Recall.	2×(0.80×0.67)/1.47 = 73%

Comparison of Evaluation Metrics

Metric	Purpose	Best Used When
Accuracy	Measures overall prediction correctness.	Dataset is balanced.
Precision	Reduces False Positives.	Incorrect positive predictions are costly.
Recall	Reduces False Negatives.	Missing positive cases is dangerous.
F1-Score	Balances Precision and Recall.	Both False Positives and False Negatives are important.

Advantages of Using Multiple Metrics

- Provides a complete evaluation of model performance.
- Identifies different types of prediction errors.
- Prevents misleading conclusions based only on Accuracy.
- Helps compare different machine learning models.
- Improves model selection for real-world applications.

Result

The classification model was successfully evaluated using Accuracy, Precision, Recall, and F1-Score. The obtained results were Accuracy = 80%, Precision = 80%, Recall = 67%, and F1-Score = 73%. These metrics demonstrate that although the model performs well overall, it misses some actual survivors. Therefore, using multiple evaluation metrics provides a more comprehensive and reliable assessment of the model than relying solely on Accuracy.

Experiment 3 — Precision, Recall and F1-Score Analysis

Aim

To understand the relationship between Precision, Recall, and F1-Score by analyzing different evaluation metric values and identifying the most balanced classification model.

Theory

Precision, Recall, and F1-Score are important performance evaluation metrics used for classification models. While Precision measures the correctness of positive predictions, Recall measures the model's ability to identify all actual positive cases. Since improving one metric may sometimes reduce the other, the F1-Score is used to provide a balanced evaluation of both metrics.

The F1-Score is calculated as the harmonic mean of Precision and Recall, making it useful when both False Positives and False Negatives are equally important.

Given Precision and Recall Values

Model	Precision	Recall	F1-Score
Model 1	0.90	0.90	0.90
Model 2	0.90	0.50	0.64
Model 3	0.60	0.90	0.72
Model 4	0.50	0.50	0.50

Interpretation of Each Model

Model 1 (Precision 0.90, Recall 0.90, F1 0.90): has both high Precision and high Recall, indicating excellent performance. It correctly identifies most positive cases while making very few incorrect positive predictions. Since both metrics are equally high, the F1-Score is also high.

Model 2 (Precision 0.90, Recall 0.50, F1 0.64): has very high Precision but low Recall. It makes very accurate positive predictions; however, it fails to identify many actual positive cases. Therefore, the overall balance of the model decreases, resulting in a lower F1-Score.

Model 3 (Precision 0.60, Recall 0.90, F1 0.72): successfully identifies most actual positive cases because of its high Recall. However, the Precision is comparatively lower, meaning that more False Positive predictions occur. The model is more suitable for applications where missing positive cases is more critical than making extra positive predictions.

Model 4 (Precision 0.50, Recall 0.50, F1 0.50): performs poorly because both Precision and Recall are low. Consequently, the F1-Score is also low, indicating weak classification performance.

Most Balanced Model

Among all four models, Model 1 is the most balanced because it has the highest Precision, highest Recall, and the highest F1-Score.

Model	Reason
Model 1	High Precision and High Recall provide the best balanced performance.

Example Calculation

Suppose a classification model has Precision = 0.80 and Recall = 0.60. The F1-Score is calculated as:

$$\text{F1-Score} = 2 \times (0.80 \times 0.60) / (0.80 + 0.60) = 0.96 / 1.40 = 0.69$$

Therefore, the model's balanced performance (F1-Score) is 69%.

Observation

- Increasing both Precision and Recall improves the F1-Score.
- If either Precision or Recall decreases significantly, the F1-Score also decreases.
- A balanced model should maintain both high Precision and high Recall.

Applications

The F1-Score is widely used in applications where both False Positives and False Negatives are important, such as:

- Medical diagnosis
- Disease detection
- Fraud detection
- Spam email filtering
- Credit card fraud detection
- Rescue and disaster management
- Customer sentiment analysis

Result

The relationship between Precision, Recall, and F1-Score was successfully analyzed. Among the given models, Model 1 achieved the highest F1-Score of 0.90, making it the most balanced classification model. The exercise demonstrated that the F1-Score effectively combines Precision and Recall into a single performance metric and is useful for evaluating machine learning classification models where both types of prediction errors must be considered.

It evaluates the balanced performance of a model, penalizing extreme differences between precision and recall.

```python
import pandas as pd
import numpy as np

# Define the classroom exercise dataset of model precision and recall pairs
data_pairs = {
    'Model': ['Model A', 'Model B', 'Model C', 'Model D'],
    'Precision': [0.9, 0.9, 0.6, 0.5],
    'Recall': [0.9, 0.5, 0.9, 0.5]
}
df_pairs = pd.DataFrame(data_pairs)

# Calculate F1-Score
df_pairs['F1-Score'] = 2 * (df_pairs['Precision'] * df_pairs['Recall']) / (df_pairs['Precision'] + df_pairs['Recall'])

# Interpret which model is most balanced
most_balanced = df_pairs.loc[df_pairs['F1-Score'].idxmax()]

print("Model Evaluation Pairs:")
print(df_pairs.to_string(index=False))
print(f"\nThe most balanced model is {most_balanced['Model']} with an F1-Score of {most_balanced['F1-Score']:.2f}")
```

✓ 0.3s Python

```
Model Evaluation Pairs:
   Model  Precision  Recall  F1-Score
Model A        0.9     0.9  0.900000
Model B        0.9     0.5  0.642857
Model C        0.6     0.9  0.720000
Model D        0.5     0.5  0.500000

The most balanced model is Model A with an F1-Score of 0.90
```